

F Prime Introduction

Timothy Canham
NASA Jet Propulsion Laboratory
February 9, 2026

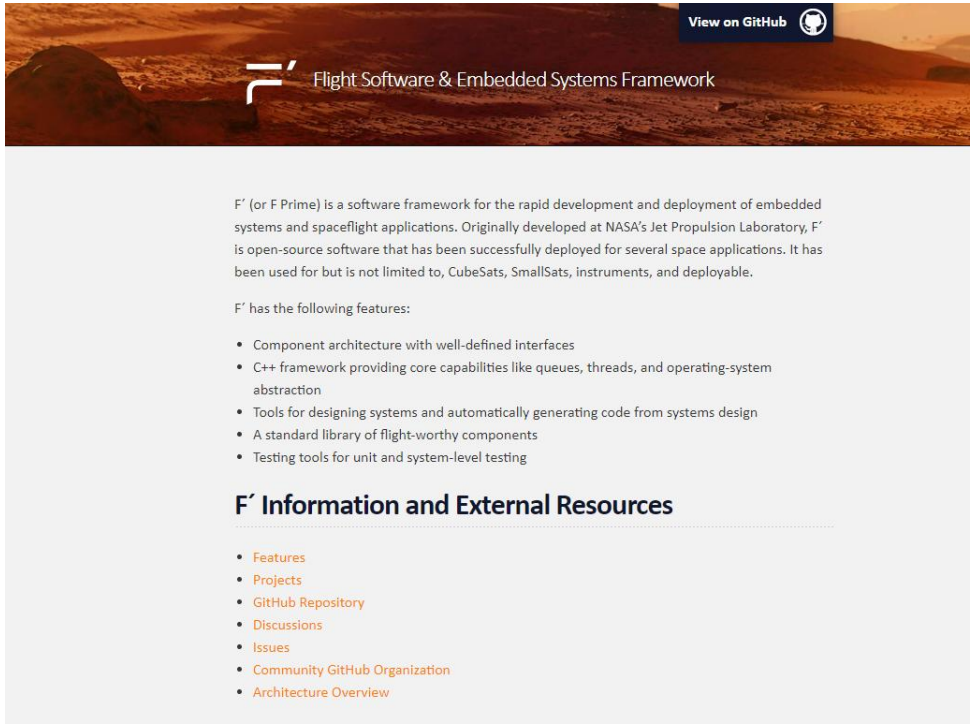
Copyright © 2026 California Institute of Technology.
Government sponsorship acknowledged.

Jet Propulsion Laboratory
California Institute of Technology

Background

- F` was developed as part of a technology task at JPL
  - Explore new flight hardware
  - Explore new software approaches
  - Targeted at smaller projects like instruments, Cubesats, and Smallsats
  - Sparser processor resources (e.g. 256Kb memory, 128K program space)
    - TI MSP430, ARM-M*, LEON3
  - Clusters of interconnected processors
- Goals were to show:

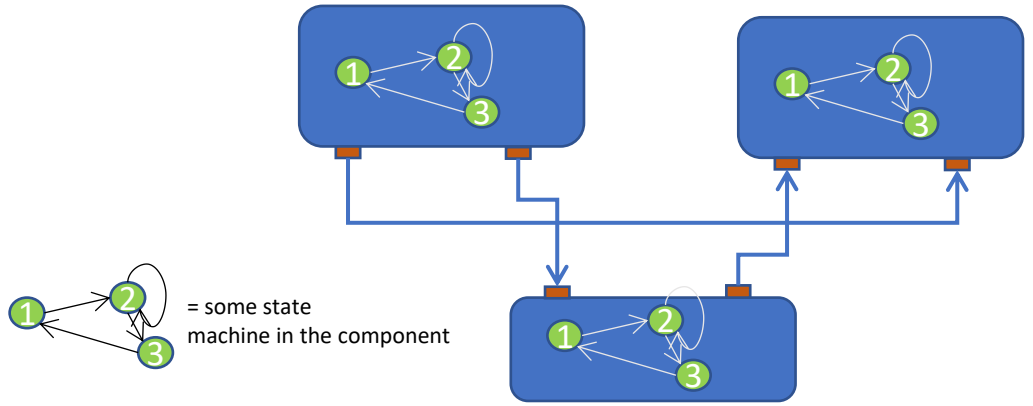
Goal	Explanation
Reusability	Frameworks and adaptations readily reusable
Modularity	Decoupled and easy to reassemble
Testability	Components easily isolated for testing
Adaptability	Should be adaptable to new contexts and bridge to inherited
Portability	Should be portable to new architectures and platforms
Usability	Should be easily understood and used by customers
Configurability	Facilities in the architecture should be scalable and configurable
Performance	Architecture should perform well in resource constrained contexts. Should be very compact.



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

F`: A Component Architecture

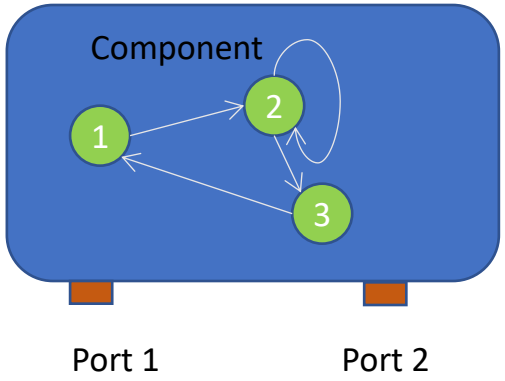
- Definition: The F´ Component Architecture is a ***design pattern*** based on an ***architectural concept*** combined with a ***software architectural framework***.
- Not just the concepts, but framework classes and tools are provided for the developer/adapter.
- Implies patterns of usages as well as constraints on usage.
- Centered around the concept of "***components***" and "***ports***"
- Uses code generation to produce code to implement common framework logic
  - Developer specifies in FPP (F Prime Prime) DSL (Domain Specific Language)
- Developer writes implementation classes to implement interfaces.



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Characteristics of Components

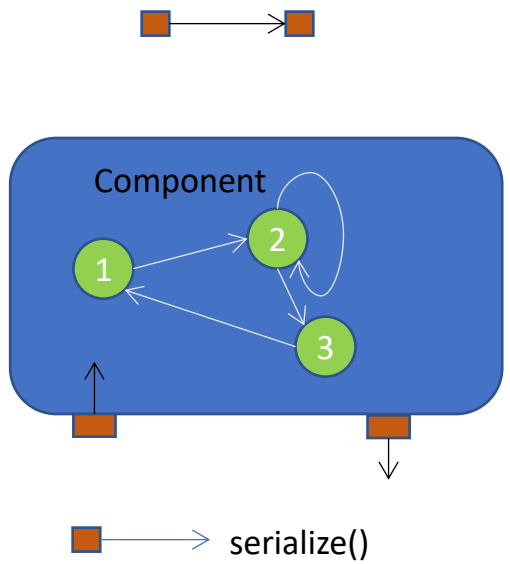
- Encapsulates behavior
- Components are ***not*** aware of other components
- Localized to one compute context
- Interfaces are via ***strongly typed*** ports
  - Ports are formally specified interfaces
  - No direct calls to other components
- Context for threads
- Executes ground commands and produces telemetry and data products



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Characteristics of Ports

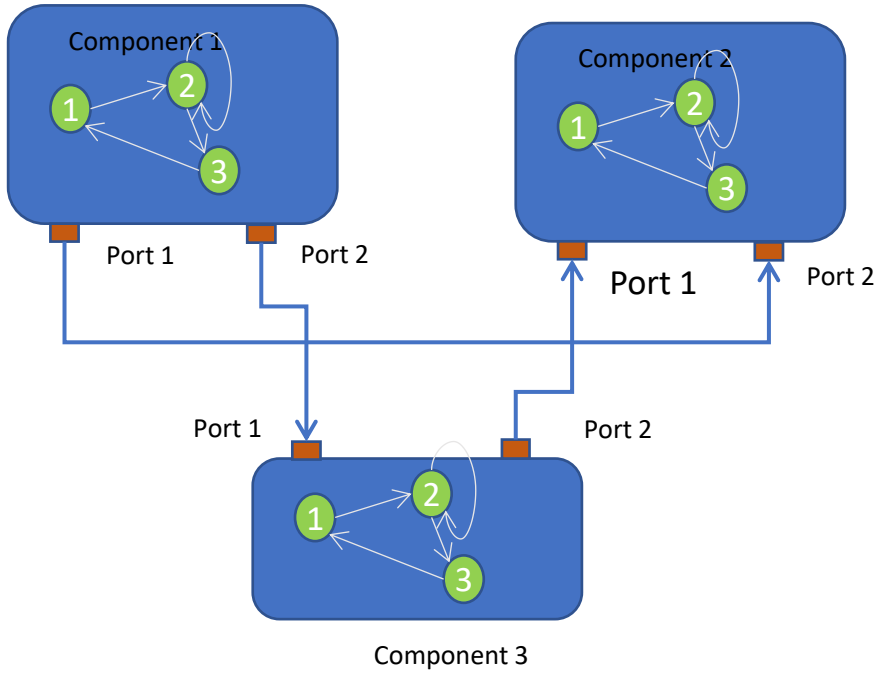
- Encapsulates ***typed*** interfaces in the architecture.
  - Think C++ class with one interface method
- Point of interconnection in the architecture.
- Ports are directional; there are input and output ports
  - Direction is direction of ***invocation***, not necessarily data flow. Ports can retrieve data.
- Ports can connect to 3 things:
  - Another typed port
    - Call is made to method on attached port
  - A component
    - Incoming port calls call component provided callback
  - A serialized port
    - Port serializes call and passes as data buffer (more to come)
- All arguments in the interface are ***serializable***, or convertible to a data buffer. There are built-in types supported by the framework; user can write custom types. (see later slides).
- Ports can have return values, but that limits use
  - Only returns data when the component has synchronous interface
  - No serializable connections since serialization passes a data buffer but does not return one (see later slides for explanation)
- Pointers/references allowed for performance reasons
- Multiple output ports can be connected to a single input port



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

A Component Topology

- Components are instantiated at run time
- They are then connected via ports into a ***Topology***, or a specific set of interconnected components
- There are no code dependencies between components, just dependencies on port interface types
- Alternate implementations can easily be swapped
  - E.g. simulation versions

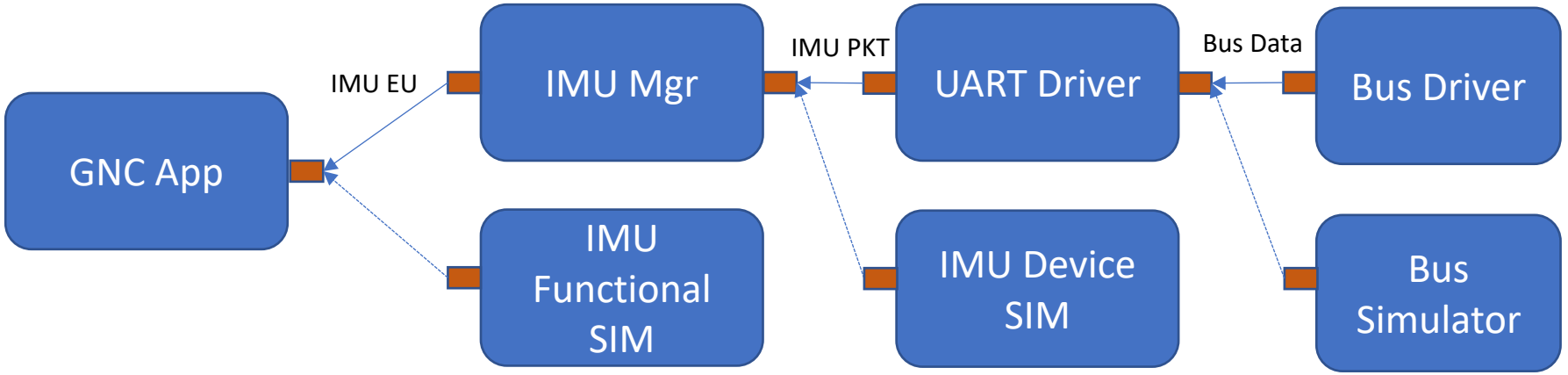


Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Deployment Variations

- F Prime allows components to be reconfigured in different configurations for testing. Ingenuity Helicopter had 11 different deployments that converged on a flight deployment that shared components
- Allowed decoupled testing and support of different venues
- Simulation versions can substitute driver/bus components for sim versions

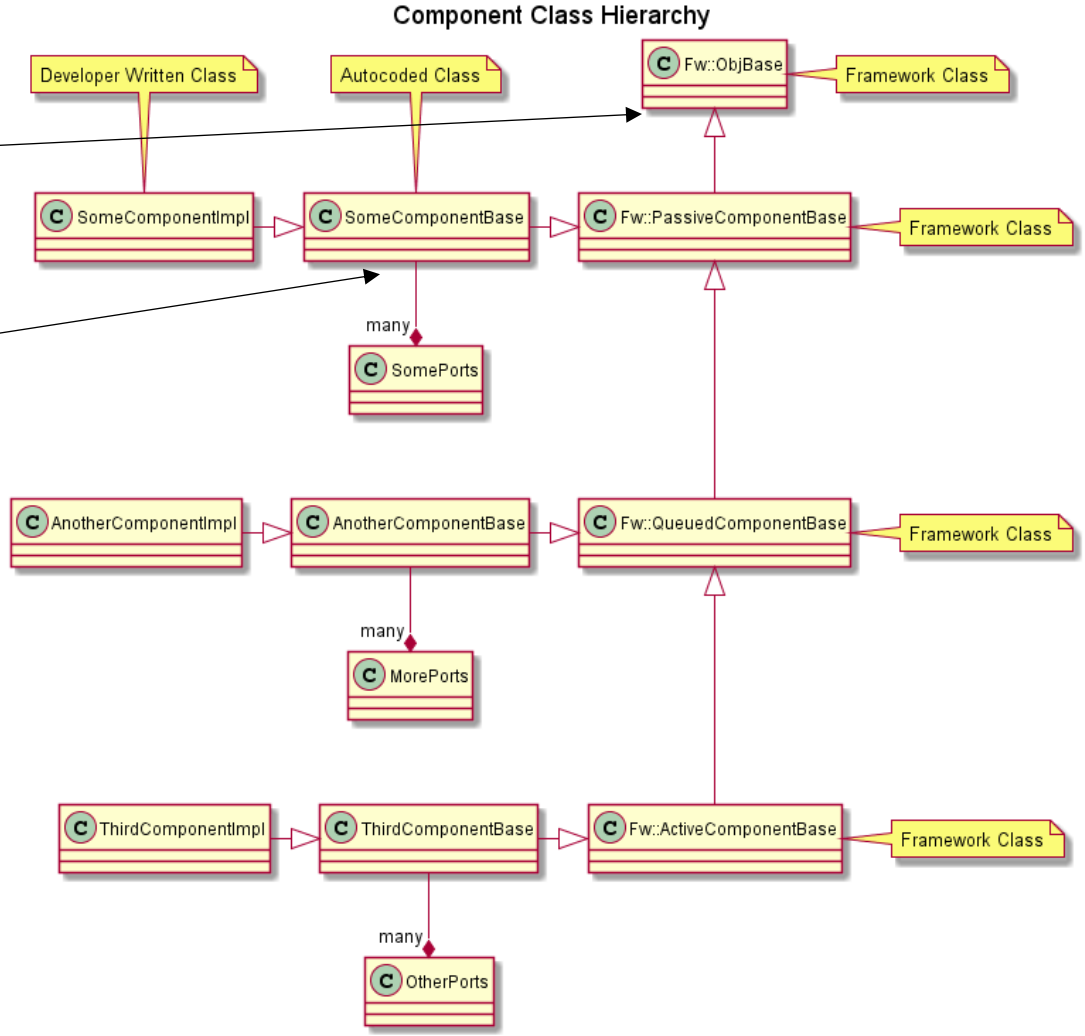
Deployment	Purpose
NAV	Flight Navigation Processor deployment
FC	Flight Flight Computer deployment
HBS	Flight Base Station deployment
COMMTEST (2x)	Radio Tester (Raspberry Pi and Snapdragon versions)
NAVCHAMBER	NAV version using flight NAV or VICON camera system in B150
NAVVICON	NAV version using only VICON for B150 (earlier version of NAVCHAMBER)
NAVOUTDOOR	Vision testing deployment for Arroyo COTS drone testing
CHAMBERBASE	GSE deployment for interfacing with GSE sensors – relayed to heli
POWERMON	GSE application to run power supplies for flat-sats
RADIORELAY	Special binary for HBS to relay helicopter data from radio to network for F Prime ground system



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Component Type Hierarchy

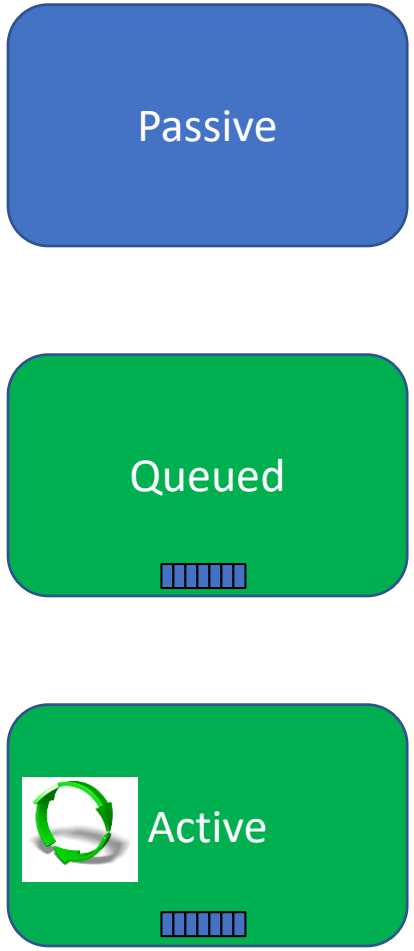
- Hierarchy consists of:
  - core framework classes
  - generated classes that implement architecture features
  - Developer written classes that implement interfaces and project-specific logic



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Component Types

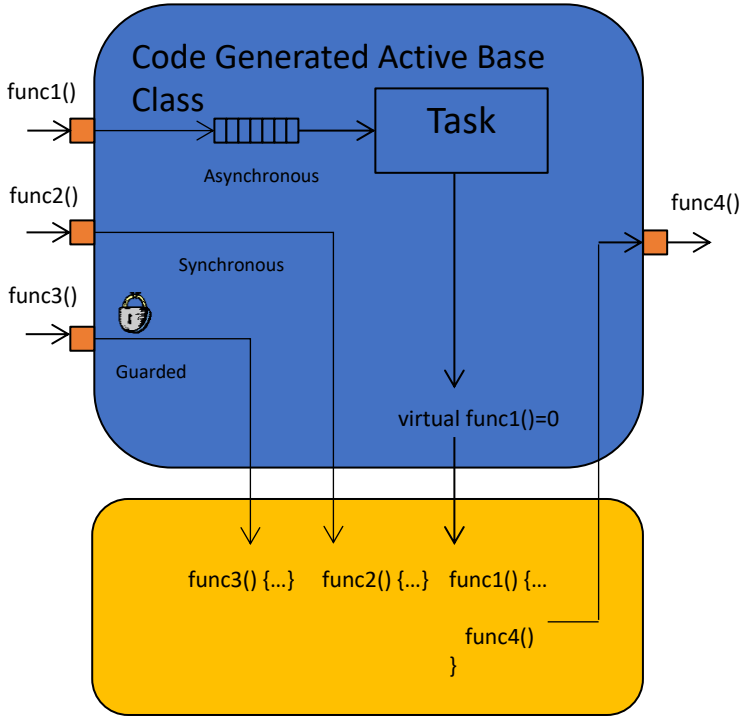
- User specifies type of component in FPP. Types are:
- Passive Component
  - No thread
  - Port interface calls are made directly to user-written handlers on thread of caller
- Queued Component
  - No thread
  - A queue is instantiated, and asynchronous port calls are serialized and placed on queue.
  - Implementation class makes call to base class to dispatch calls to user handlers for asynchronous ports
    - Can be made from any implementation class function
    - Thread of execution provided by caller to a synchronous port
- Active Component
  - Component has thread of execution as well as queue
  - Thread dispatches port calls from queue as it executes based on thread scheduler
- For all component types, calls to output port are on thread of implementation functions
  - Thread making call is dependent on port characteristic (see next slide)



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Port Characteristics

- The way incoming port calls are handled is specified by the ***component*** FPP.
- Input ports can have three attributes:
  - Synchronous – port calls directly invoke handlers without passing through queue
  - Guarded – port calls directly invoke handlers, but only after locking a mutex shared by all guarded ports in component
  - Asynchronous – port calls are placed in a queue and dispatched on thread emptying the queue.
- A passive component can have synchronous and guarded ports, but no asynchronous ports since there is no queue. Calls execute on the thread of the calling component.
- A queued component can have all three port types, but it needs at least one synchronous or guarded port to unload the queue and at least one asynchronous port for the queue to make sense.
- An active component can have all three varieties but needs at least one asynchronous port for the queue and thread to make sense.
- Developer needs to be aware of how all the different call kinds interact (e.g. reentrancy)
- Output ports are invoked by calling generated base class functions from the implementation class.

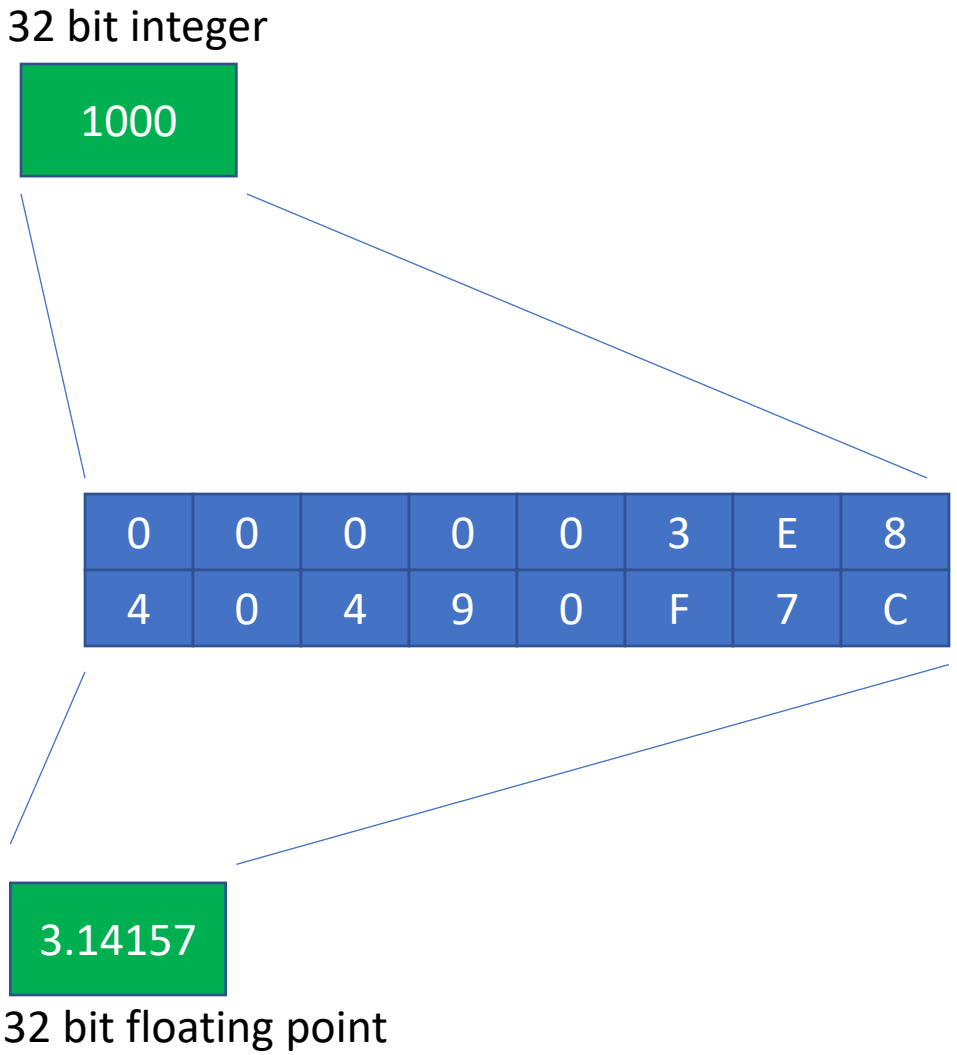


Developer Written Implementation Class

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Serialization

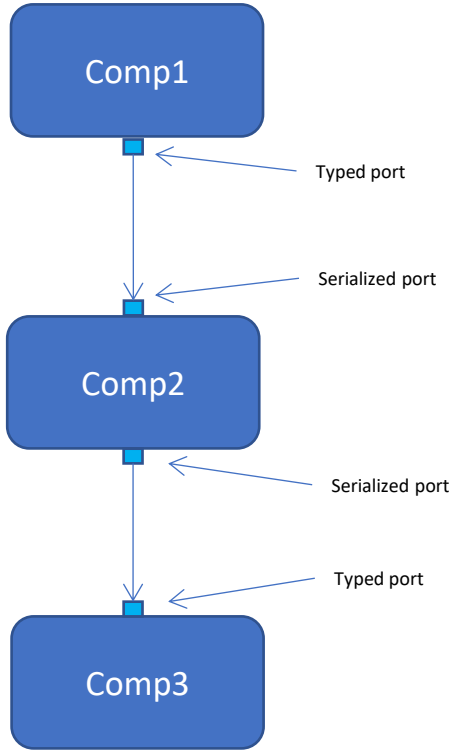
- Serialization is a ***key*** concept in the framework
- Definition: Taking a specific set of typed values or function arguments and converting them in an architecture-independent way into a data buffer
- Port calls, commands and their arguments are serialized and placed on message queues in components
- Command arguments and telemetry values are passed and encoded/decoded into this form
- Components that store and pass data can use this form and not require knowledge of underlying types
- User can define arbitrary interface argument types and framework automatically serializes
- User can define complex types in FPP and code generator will generate classes that are serializable for use internally and to and from ground software
- Data is stored in Big Endian order



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Serialization Ports

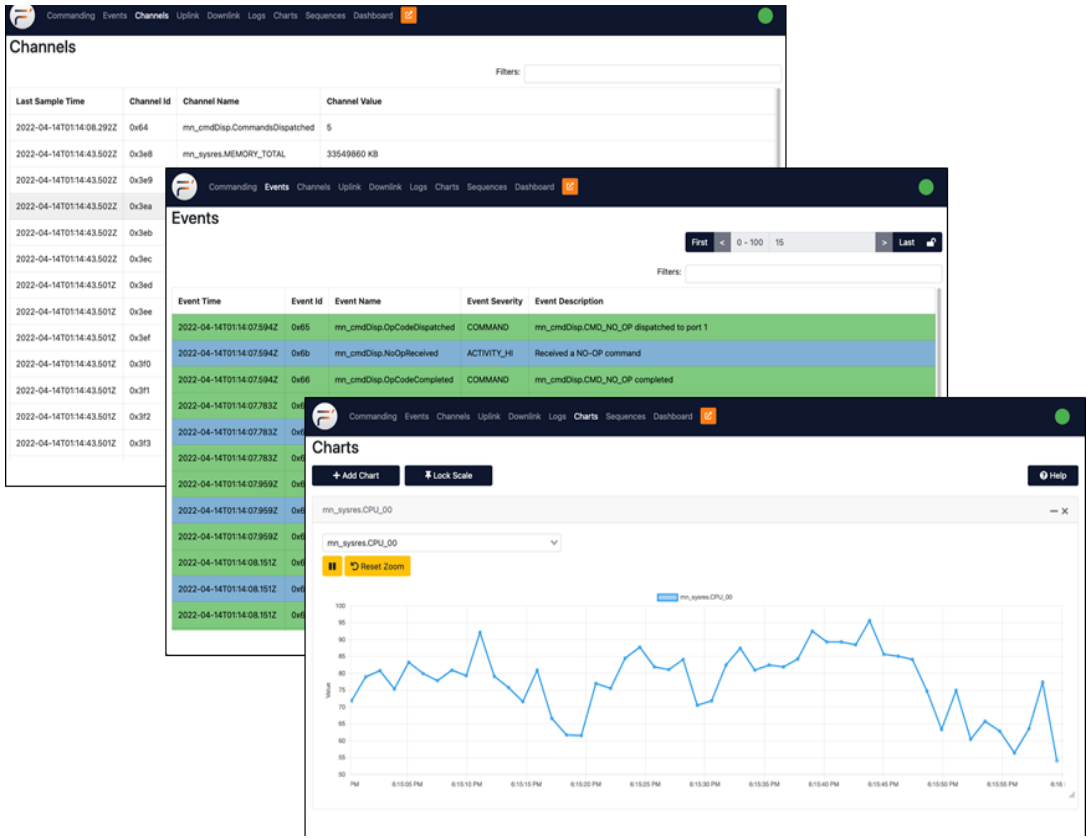
- A special optional port that handles serialized buffers
- Takes as input a serialized buffer when it is an input port, and outputs a serialized buffer when it is an output port.
- Can be connected to *any* typed port (almost).
  - For input port, calling port detects connection and serializes arguments
  - For output port, serialized port calls interface on typed port that deserializes arguments
  - ***Not*** supported for ports with return types
- Useful for generic storage and communication components that don't need to know type
  - Allows design and implementation of C&DH (command and data handling) components that can be reused.



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Commands, Telemetry, Events, Parameters and Data Products

- The code generator provides a method of implementing commands, telemetry, events (AKA JPL EVRs), parameters, and data products
- Component FPP specifies arguments and types.
- Data for service is passed in serialized form.
- The code generator parses arguments and types and generates code to convert arguments to serialized data or vice versa.
- Calls implementation functions with deserialized arguments (commands) or provides base class functions to implement calls (telemetry, events, parameters and data products)
- Code generator uses port types that are specified in XML themselves. These ports can be then used in components that provide interfaces for transporting or storing data for those services.
    - Since data is handled in serialized form, don’t need to know specific argument types



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Commands

- Commands are sent from the ground system to the spacecraft to initiate activities
- Component command FPP specifies:
  - Opcode, mnemonic, and arguments
    - Arguments can be any built-in type or external XML complex type
    - Complex type can be a single argument
    - Can define enumerations
  - Synchronization attribute
    - Sync, async, or guarded
    - Same meaning as ports
      - Async can specify message priority
- Implementation class implements function for each command
  - Framework code deserializes arguments from argument data buffer
- User adds ports for registering commands, receiving commands and reporting an execution status.

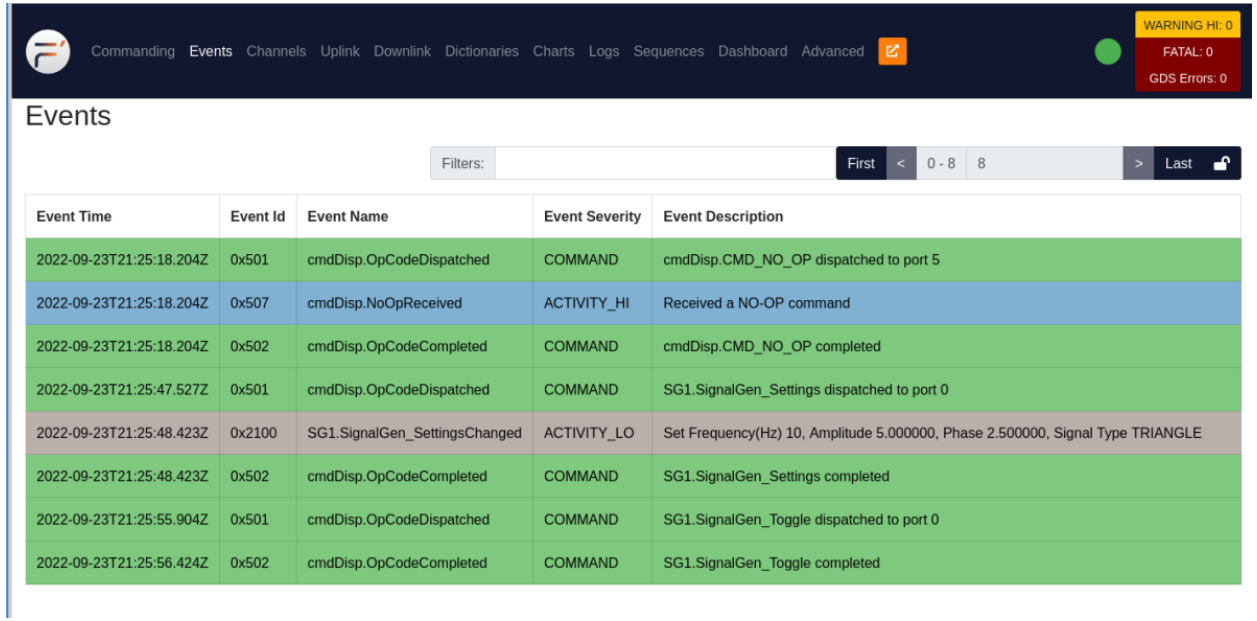


Ingenuity Uplink Team

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Events

- Events are asynchronous notifications of spacecraft activity
- Component event FPP specifies:
  - ID, name, severity and arguments
    - Arguments can be any built-in type or XML complex type
    - Complex type can be a single argument
    - Can define enumerations
  - Format specifier string
    - Used by ground software and optional on-board console to display message with argument values
    - Follows C format specifier syntax
- Code generated base class provides function to call for each event with typed arguments
  - Provides stronger type checking at compile time than MER/MSL EVR macros
  - Called by implementation class
- Code generator automatically adds ports for retrieving time tag and sending event
  - Two independent ports for sending events
    - A binary version with serialized arguments for transport to ground software
    - A text port that sends a string version of the event (using the format specifier) that can be sent to a console
      - Can be globally disabled via architecture configuration macros to save execution time and code space

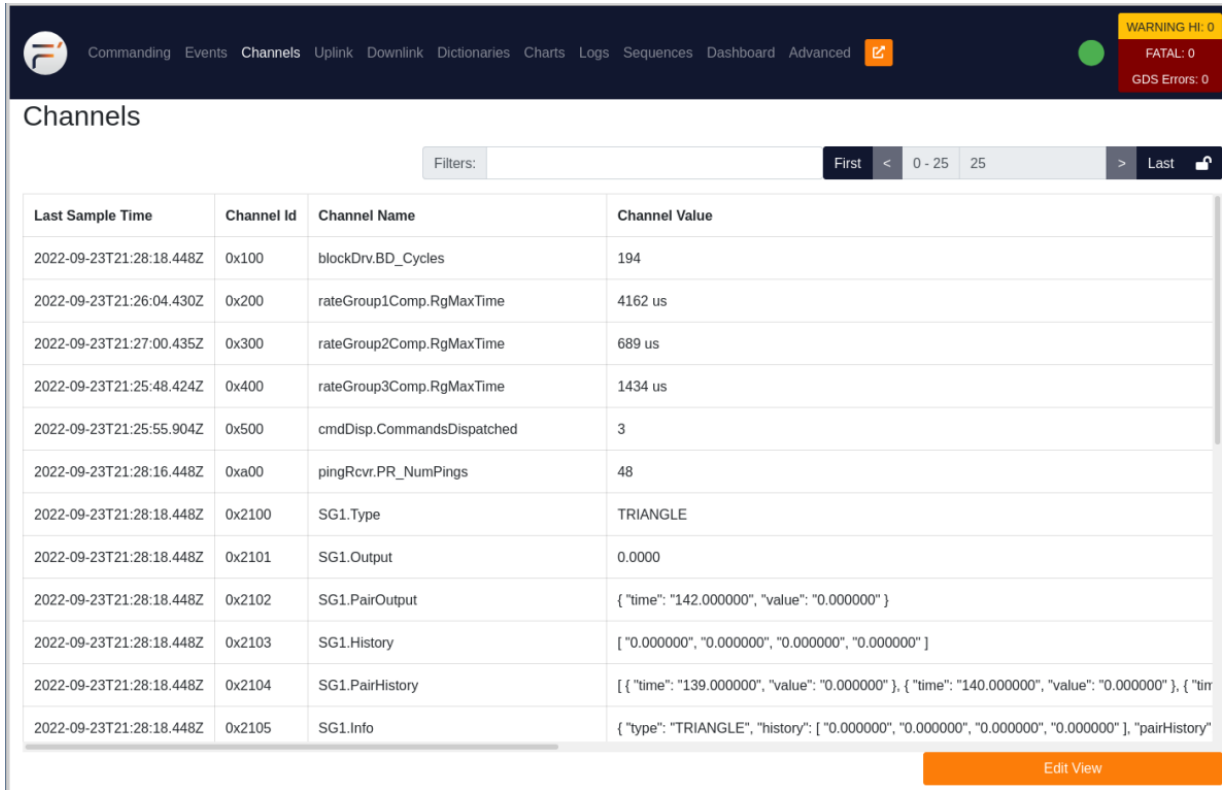


Event Time	Event Id	Event Name	Event Severity	Event Description
2022-09-23T21:25:18.204Z	0x501	cmdDisp.OpCodeDispatched	COMMAND	cmdDisp.CMD_NO_OP dispatched to port 5
2022-09-23T21:25:18.204Z	0x507	cmdDisp.NoOpReceived	ACTIVITY_HI	Received a NO-OP command
2022-09-23T21:25:18.204Z	0x502	cmdDisp.OpCodeCompleted	COMMAND	cmdDisp.CMD_NO_OP completed
2022-09-23T21:25:47.527Z	0x501	cmdDisp.OpCodeDispatched	COMMAND	SG1.SignalGen_Settings dispatched to port 0
2022-09-23T21:25:48.423Z	0x2100	SG1.SignalGen_SettingsChanged	ACTIVITY_LO	Set Frequency(Hz) 10, Amplitude 5.000000, Phase 2.500000, Signal Type TRIANGLE
2022-09-23T21:25:48.423Z	0x502	cmdDisp.OpCodeCompleted	COMMAND	SG1.SignalGen_Settings completed
2022-09-23T21:25:55.904Z	0x501	cmdDisp.OpCodeDispatched	COMMAND	SG1.SignalGen_Toggle dispatched to port 0
2022-09-23T21:25:56.424Z	0x502	cmdDisp.OpCodeCompleted	COMMAND	SG1.SignalGen_Toggle completed

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Telemetry

- Telemetry is data periodically sampled and emitted
- Component telemetry FPP specifies channels that have:
  - ID, name, and data type
    - Data type can be any built-in type or external XML complex type
    - Can define enumerations
  - Format specifier string
    - Used by ground software and optional on-board console to display message with argument values
- Code generated base class provides function to call for each channel with typed argument
  - Called by implementation class
- Code generator automatically adds ports for retrieving time tag and sending channelized data



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

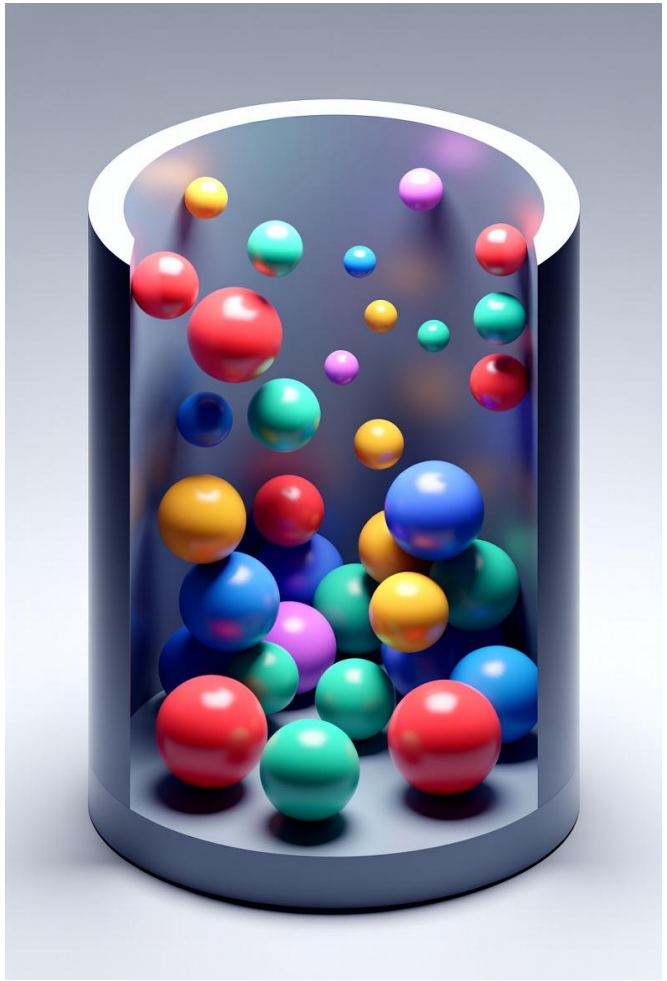
Parameters

- Parameters are a means of storing non-volatile state
  - Allows modification of software behavior without a software update
    - Ex. Thermal alarm limits
- Component FPP specifies parameters that have:
  - ID, name, and data type
    - Data type can be any built-in type or external XML complex type
    - Can define enumerations
  - Optional default value
    - In the event the parameter cannot be retrieved, assigns default value to parameter
- Code generator automatically adds port for retrieving parameters
- During initialization, a public method in the class is called that retrieves the parameters and stores copies locally
  - Can be called again if parameter is updated
- Code generated base class provides function to call for each parameter to retrieve stored copy
  - Implementation class can call whenever parameter value is needed
- Framework provides code generation to manage, but projects must write specific storage component
  - Basic file-based storage provided by F Prime core component

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Data Products

- Data Products are data sets generated on board
- They can be used for payload data, high-rate logs, etc.
- Data product *records* are defined as a data type in FPP
- Users define typeless *containers* for the records in FPP
- Users pre-allocate memory with enough space to contain the number of records needed in a *container*
- Users fill the container with records
  - Records can be of different types
- When complete, the container is sent to storage for later downlink
  - Users can develop compressors or other processors as a plug-in for the data prior to writing with *Svc/DpWriter*
- The container is assigned a downlink priority
- The *Svc/DpCatalog* component will downlink the data product files in priority order
  - Priority can be changed dynamically to reprioritize



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Code Scaling

- Framework code is very compact
- Generated code is also compact
  - Demo application for Arduino is about 32K RAM/64K Flash
- Native type sizes can be configured
  - e.g. some microcontrollers have 16-bit/8-bit only support
- Features can be added or removed depending on resources
  - Object naming
  - Port execution tracing
  - Serialization of ports
    - Single node systems don't really need
    - This is not data serialization but the use of serialized ports
  - Object naming/registry
  - Component connection tracing
  - Text logging
- For very compact processors with no OS, developers can choose non-active components



This Photo by Unknown Author is licensed under CC BY-NC

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

F Prime Core Components

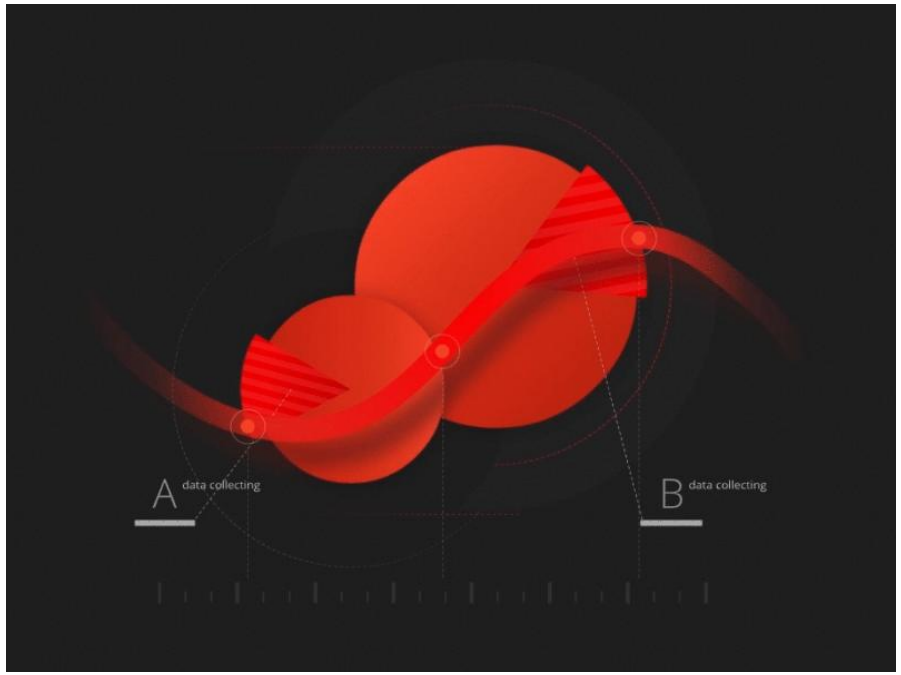
- The F Prime repository has a set of tested and flight-proven components to do common spacecraft functions

Component	Description
Command Dispatcher	Dispatches uplinked commands to components
Command Sequencer	Executes a set of commands uplinked as a file
Telemetry Processing	Gets sampled telemetry from components for downlink
Event Logging	Gets asynchronous event notifications from components for downlink
File Uplink/Downlink	Send and receive files
File Manager	File management including listing, moving and deleting files
Parameter Database	Database to store non-volatile parameter values for components
Value Database	Allows components to exchange run-time data values
Rate Group Execution	Allows a set of components to run sequentially at a certain rate
Uplink/Downlink Framing	Generic components for framing data for telecommunications
Buffer Management	Pre-allocates a set of buffers for dynamic memory requirements for components

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Recent New Features

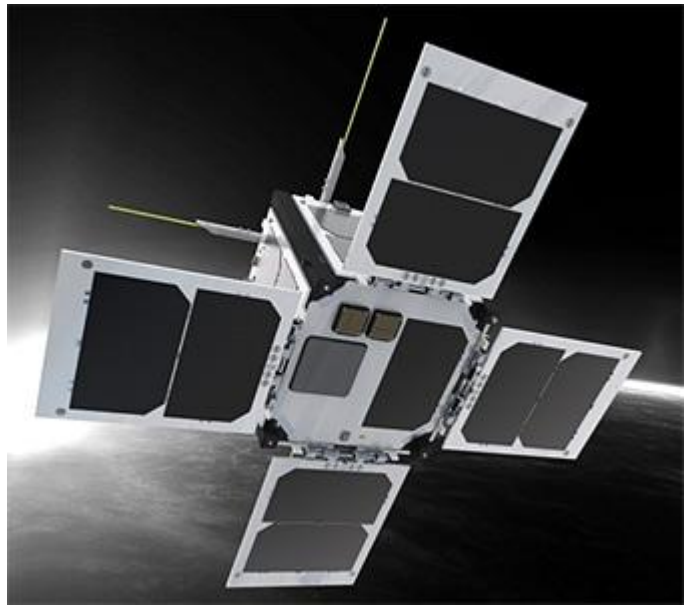
- Data products
  - Define and log structured data
  - Data catalog to automatically prioritize and downlink structured data logs
  - Ground tools to decode the data
- Advanced sequencing
  - Conditional based on:
    - Command status
    - Local/global variables
    - Telemetry (PolyDb or ground)
  - Looping
  - Sequence arguments



Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.

Summary

- F Prime provides flight software-in-a-box
- Projects benefit from:
  - A ready-to-go framework
  - Flight heritage
  - Open-source support
  - Designed for embedded processors
  - Unit test and system test support
  - Basic ground system



Georgia Tech GT-1

Copyright © 2026 California Institute of Technology. Government sponsorship acknowledged.